

Queue and Worker Model

Many tools invoked by the RAGenius Execution Subsystem (e.g. video generation, document rendering, external API calls) are long-running or compute-intensive. To avoid blocking HTTP requests and to increase scalability, the subsystem should support asynchronous execution using a queue and worker model. This document outlines a suggested approach.

Objectives

1. **Non-blocking:** Return control to the API caller immediately while the execution proceeds in the background.
2. **Scalability:** Distribute work across multiple worker processes.
3. **Resilience:** Persist the queue so that work is not lost if a worker crashes.
4. **Observability:** Track the status of queued jobs and provide progress updates.
5. **Control:** Support cancellation, retries, and priority.
6. **Fairness:** Prevent one application or tenant from starving the queue.

Architecture

1. **Task Queue:** Use a durable queue system (e.g. RabbitMQ, Apache Kafka, Redis Streams, or PostgreSQL advisory locks) to store tasks. Each task corresponds to an execution or a step that needs asynchronous processing.
2. **Workers:** Independent processes or threads that consume tasks from the queue and run the workflow steps. Workers update execution status in the database and may enqueue follow-up tasks.
3. **Scheduler:** A component that monitors executions and enqueues tasks (e.g. when a request arrives, or when a step completes and the next step is asynchronous).
4. **API Layer:** Submits tasks, returns immediately (e.g. with `202 Accepted`), and exposes endpoints to poll or subscribe to status updates.

Task Structure

A task message could include:

```
{
  "execution_id": "exec_123",
  "step_id": "generate_video",
  "attempt": 1,
  "payload": {
    "prompt": "Explain RAG",
    "duration": 30,
    "context": "..."
  },
}
```

```
"scheduled_at": "2026-05-09T12:00:00Z"
}
```

- `execution_id` and `step_id` identify the work.
- `attempt` counts retries.
- `payload` contains the input for the tool call.
- `scheduled_at` allows delayed execution (for backoff).

Worker Responsibilities

1. **Fetch task:** Block or poll the queue for new tasks.
2. **Load context:** Retrieve the current execution and step from the database. Ensure the execution is still in `running` status.
3. **Run step:** Invoke the tool or service call. Enforce timeouts and permission checks.
4. **Handle result:** Update the step and execution status. Enqueue the next step(s) if needed. Run compensation steps if failure triggers a saga.
5. **Retry or fail:** On recoverable errors (`tool` , `timeout` , `external_api`), reschedule the task with an incremented `attempt` and an exponential backoff. On unrecoverable errors (`validation` , `permission` , `workflow`), mark the execution failed.
6. **Acknowledge:** Remove the task from the queue only after the step result has been persisted, ensuring at-least-once semantics.

API Considerations

Submitting Executions

- `POST /v1/executions` : Should enqueue an execution task and return immediately. The response includes an `execution_id` and a `status` of `queued` or `pending_confirmation` .
- The execution context is created in the database before the task is enqueued.

Polling Status

- `GET /v1/executions/:execution_id` should return the current status and result (if finished). For queued or running executions, clients should poll or subscribe to updates.
- Consider implementing WebSocket or Server-Sent Events (SSE) endpoints for real-time notifications.

Cancellation

- Provide a `POST /v1/executions/:execution_id/cancel` endpoint. It should set the execution status to `cancelling` or `cancelled` . Workers should check for cancellation before starting a step and stop processing if cancelled.
- If a tool call is in progress and supports cancellation, attempt to cancel it (e.g. abort HTTP request). Otherwise, allow the step to finish but mark subsequent steps as cancelled.

Queue Implementation Options

- **RabbitMQ:** Provides reliable queues with acknowledgement, routing, and priority. Suitable for moderate to high throughput.
- **Apache Kafka:** Provides a distributed commit log. Suitable for high-throughput pipelines. Requires careful consumer management for exactly-once semantics.
- **Redis Streams:** Simpler to operate but may require additional tooling to ensure durability (e.g. Redis persistence).
- **PostgreSQL:** Use advisory locks and a `tasks` table. Simpler to deploy but may not scale to very high throughput.
- Choose the system that matches your operational expertise and throughput requirements.

Scaling and Fairness

- Run multiple worker instances to increase throughput. Use auto-scaling to adjust to load.
- Use partitioning or sharding to isolate tasks by tenant or application, preventing a noisy tenant from starving others.
- Implement priority queues if certain skills or tasks must run sooner than others.
- Limit the maximum number of concurrent side-effecting tool calls to avoid overwhelming external providers.

Monitoring

Monitor and emit metrics such as:

- Queue length and latency.
- Number of tasks processed per minute.
- Execution status transitions (queued → running → completed).
- Worker success/failure rate.
- Average and p95 step duration.
- Retry counts and backoff delays.
- Cancellation and abandonment counts.

These metrics help identify bottlenecks and ensure SLA compliance.

Conclusion

A queue and worker model is essential for handling long-running and high-throughput workflows. It decouples API responsiveness from execution latency, enables horizontal scaling, and provides the foundation for reliability and observability. Future work could include dynamic worker provisioning, more granular task sharding, and integration with serverless functions for cost-efficient scaling.
